

Basics of Automation and Control

Industrial Processes, Control Elements, Actuators & PID

Amit Kumar Singh
Assistant Professor (ECE)
School of Naval Architecture and Ocean Engineering
Indian Maritime University Visakhapatnam Campus, Sabbavaram Mandal
Visakhapatnam, Andhra Pradesh – 531 035

amitkumarsingh@imu.ac.in

September 24, 2026

- 1 Industrial Processes
- 2 Automation of Industrial Processes
- 3 Elements of a Control System
- 4 Pneumatic Control Systems
- 5 Hydraulic Control Systems
- 6 Electrical Control Systems
- 7 Valves
- 8 Actuators
- 9 Stepper Motors
- 10 PID Controllers
- 11 Tuning of PID Controllers

What is an Industrial Process?

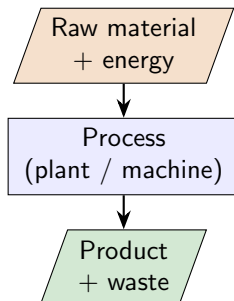
Definition

An **industrial process** is a planned sequence of **physical, chemical, or mechanical** operations that converts **raw materials** (input) into useful **products** (output) using energy, machines and people.

- It follows the simple cycle:

RAW MATERIAL → **PROCESS** → **PRODUCT**

- Examples: refining crude oil, generating power, bottling drinks, welding a ship hull.
- Every process has quantities that must be *kept at the right value* — temperature, pressure, flow, level, speed, position.



Types of Industrial Processes

Continuous

Runs non-stop, 24×7

- Material flows without interruption.
- Variables are *analog* (temperature, flow, level).
- Examples: oil refinery, power plant, water treatment.

Batch

Fixed quantity at a time

- A recipe is run step by step on one lot.
- Mix $\rightarrow$ heat $\rightarrow$ hold $\rightarrow$ empty.
- Examples: medicines, paint, food, beverages.

Discrete

Individual countable parts

- Parts are made, moved and assembled.
- Variables are mostly *ON/OFF* (part present, cylinder out).
- Examples: car assembly, packaging, shipbuilding.

Continuous & batch $\rightarrow$ mainly **process control** (PID loops)

Discrete $\rightarrow$ mainly **sequence / logic control** (PLC)

Process Variables and Signals

Variable	Unit	Typical sensor	Example
Temperature	°C, K	Thermocouple, RTD (Pt100)	boiler, engine cooling
Pressure	bar, Pa	Strain-gauge / piezo transmitter	steam line, hydraulics
Flow	m ³ /h, L/min	Orifice + DP cell, magnetic meter	fuel, cooling water
Level	m, %	Float, ultrasonic, DP transmitter	ballast tank, boiler drum
Speed / position	rpm, mm	Tachogenerator, encoder, LVDT	shaft, rudder angle

Standard instrument signals

4–20 mA (current loop), **0–10 V**, pneumatic
0.2–1.0 bar (3–15 psi).

Why 4 mA and not 0 mA?

A “live zero”: 0 mA means a **broken wire**, not a zero reading.

Converting a 4–20 mA Signal: Worked Example

Linear scaling

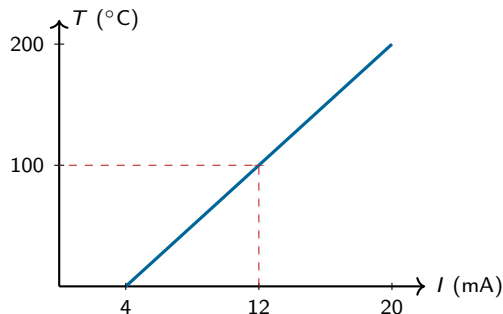
$$PV = LRV + \frac{I - 4}{20 - 4} \times (URV - LRV)$$

LRV / URV = lower / upper range value of the transmitter.

Problem: a temperature transmitter is ranged 0–200 °C. The controller reads **12 mA**. What is the temperature?

Solution:

$$T = 0 + \frac{12 - 4}{16} \times 200 = \frac{8}{16} \times 200 = \boxed{100^\circ\text{C}}$$



What is Automation?

Automation

Automation is the use of **control systems** (sensors, controllers, actuators, computers) to operate machines and processes with **minimum human intervention**.

Why automate?

- Higher **productivity** and consistent **quality**.
- **Safety** — remove people from hazardous areas.
- Lower operating cost, less waste & energy.
- Accurate data logging and remote monitoring.

Limitations

- High initial (capital) cost.
- Needs skilled maintenance staff.
- Less flexible for one-off jobs.
- Cyber-security & single-point failures.

On a ship: engine-room automation lets an *unmanned machinery space* (UMS) run safely at night.

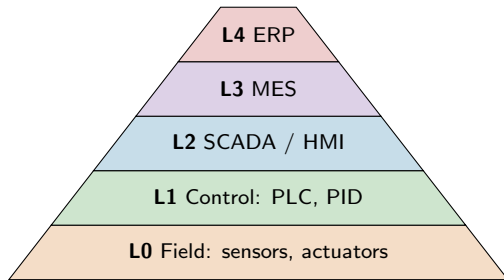
Types of Automation

Type	Idea	Volume / variety	Example
Fixed (hard)	Sequence fixed by the equipment design	very high volume, one product	bottling line, transfer machine
Programmable	Sequence changed by a new program	medium volume, batches	CNC machine, PLC-driven press
Flexible	Changes product with no lost time	medium volume, many variants	robotic welding cell, FMS

Key idea

Moving down the table, the system becomes **more adaptable** but also **more complex** and costly per unit. The choice depends on *how many* and *how many different* products we make.

The Automation Pyramid (Levels of Control)



Going **up**: slower, more data, business view.

Going **down**: faster, real-time, closer to the machine.

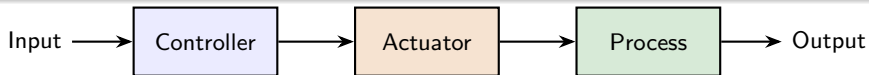
- **Field**: measure and act on the process (ms).
- **Control**: run the control logic / loops (ms–s).
- **Supervisory**: operator screens, alarms, trends (s).
- **MES / ERP**: production planning, orders, business (hours–days).

This course concentrates on **Levels 0 and 1**.

Open-Loop vs. Closed-Loop Control

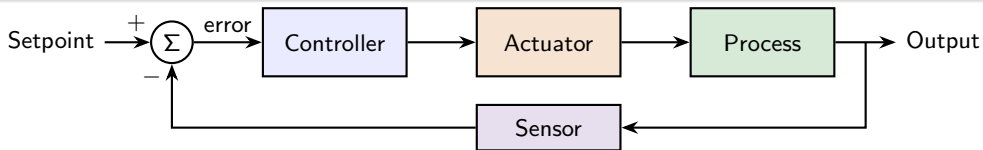
Open loop

Output is **not** measured — the controller acts blindly (e.g. a toaster timer, a washing-machine cycle).

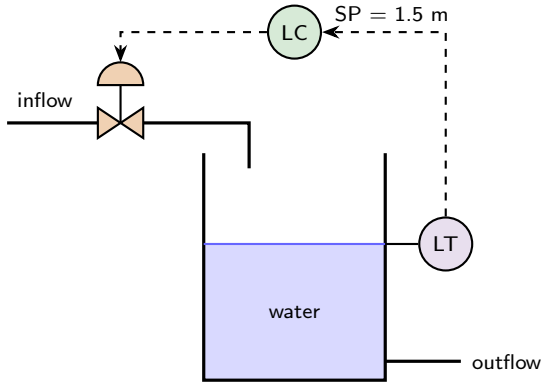


Closed loop (feedback)

Output is **measured** and compared with the desired value; the **error** drives the controller (e.g. AC thermostat, autopilot).



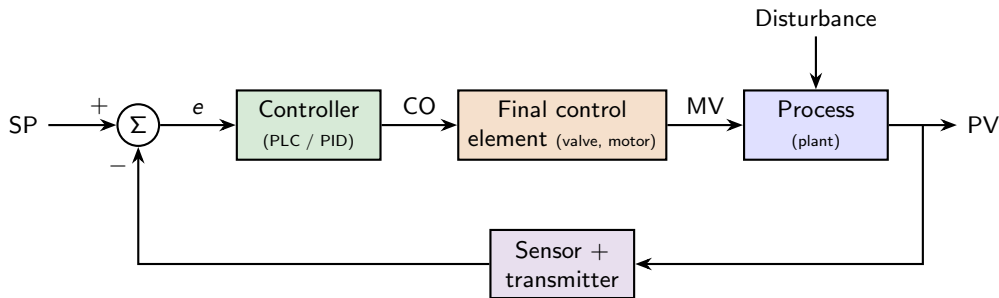
A Closed-Loop Example: Tank Level Control



- 1 **LT** (level transmitter) measures the level and sends 4–20 mA.
- 2 **LC** (level controller) compares it with the **setpoint**.
- 3 If the level is low, LC **opens** the inlet control valve more; if high, it **closes** it.
- 4 The loop repeats continuously — level stays at 1.5 m even if outflow changes (*disturbance*).

Dashed lines = instrument signals; solid lines = process pipes (ISA-5.1 style).

Building Blocks of Every Control Loop



SP	Setpoint (desired value)	CO	Controller output
PV	Process variable (measured)	MV	Manipulated variable
e	Error = $SP - PV$		(e.g. flow through valve)

Three Families of Control Systems

Pneumatic

Medium: compressed air

- 6–8 bar working pressure.
- Fast, clean, cheap, spark-free.
- Light loads, simple motion.

Hydraulic

Medium: pressurised oil

- 100–350 bar working pressure.
- Very large forces, stiff & precise.
- Steering gear, cranes, presses.

Electrical

Medium: electric current

- Relays, contactors, PLCs, drives.
- Easy to transmit & to program.
- Motors, solenoids, heaters.

Modern machines are usually **electro-pneumatic** or **electro-hydraulic**:
electrical signals decide, **fluid** power does the heavy work.

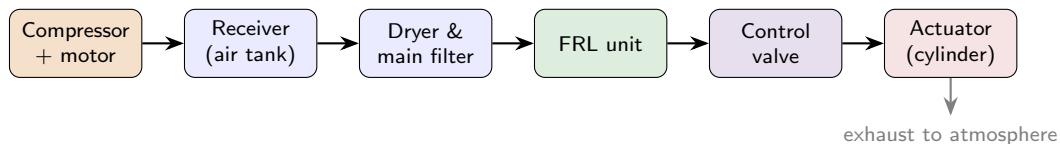
Comparing Pneumatic, Hydraulic & Electrical Systems

Feature	Pneumatic	Hydraulic	Electrical
Force	low–medium	very high	low–high
Speed	high	medium	high
Position accuracy	poor (air compresses)	good (oil is stiff)	excellent (servo/stepper)
Overload	safe — just stalls	safe with relief valve	needs protection
Cleanliness	clean, exhaust to air	leaks are messy, fire risk	clean
Cost	low	high	medium
Typical use	packaging, clamping	steering gear, presses	conveyors, robots, pumps

Rule of thumb

Big force? → hydraulics. **Fast, light, cheap?** → pneumatics. **Precision?** → electrical.

Elements of a Pneumatic System



- **Compressor** — raises air to 6–10 bar; **receiver** stores it and smooths pulsations.
- **Dryer / filter** — removes water and dirt (moisture causes corrosion & valve sticking).
- **FRL** = **F**ilter + **R**egulator (sets constant pressure) + **L**ubricator (oil mist).
- **Control valves** — decide *where*, *how much* and *how fast* air flows.
- **Actuators** — turn air pressure into motion (cylinders, air motors).

Pneumatics: Properties of Compressed Air

Advantages

- Air is free and available everywhere.
- Easily stored and piped over distance.
- Explosion-proof — no sparks (paint shops, mines, tankers).
- Actuators can stall under overload without damage.
- Very high speeds (up to 1–2 m/s for cylinders).

Disadvantages

- Air is **compressible** → uneven, “spongy” motion.
- Force limited (practical limit ≈ 30 kN).
- Exhaust noise — silencers needed.
- Energy-inefficient (compressor losses, leaks).
- Air must be clean and dry.

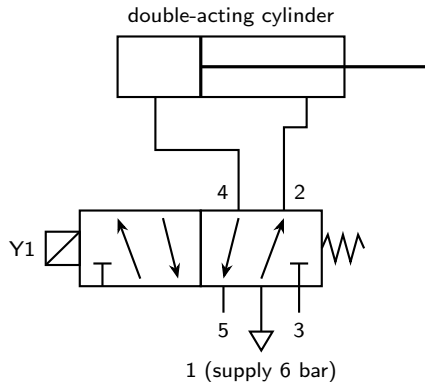
Units to remember

1 bar = 10^5 Pa = 0.1 MPa $\approx$ 14.5 psi

1 MPa = 1 N/mm²
atmospheric

Gauge pressure = absolute –

Example Pneumatic Circuit: Double-Acting Cylinder



- **Valve at rest** (spring side): port 1 → 2 feeds the rod side, port 4 → 5 exhausts the cap side ⇒ cylinder **retracts**.
- **Solenoid Y1 energised**: valve shifts, 1 → 4 feeds the cap side, 2 → 3 exhausts ⇒ cylinder **extends**.
- The PLC only switches a 24 V coil — the air does the work.

Reading a valve symbol

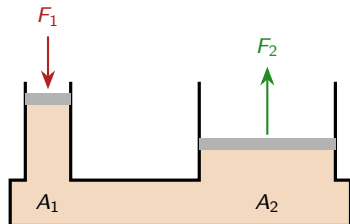
Each **square** = one switching position. The square touching the port lines is the *current* (rest) position.

Pascal's Law: The Heart of Hydraulics

Pascal's law

Pressure applied to a confined fluid is transmitted **equally in all directions**.

$$p = \frac{F_1}{A_1} = \frac{F_2}{A_2} \implies F_2 = F_1 \frac{A_2}{A_1}$$



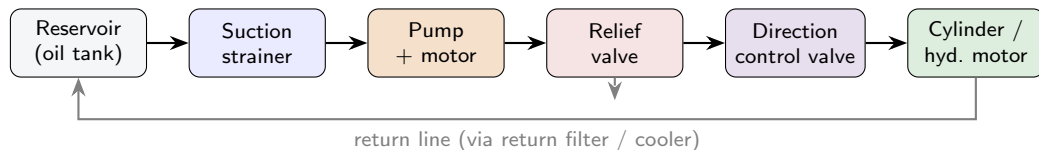
Example: $F_1 = 100 \text{ N}$ on $A_1 = 10 \text{ cm}^2$;
 $A_2 = 200 \text{ cm}^2$.

$$p = \frac{100}{10 \times 10^{-4}} = 10^5 \text{ Pa} = 1 \text{ bar}$$

$$F_2 = 100 \times \frac{200}{10} = \boxed{2000 \text{ N}}$$

A $20\times$ force gain — but the big piston moves $20\times$ *less* distance (energy is conserved).

Elements of a Hydraulic System



- **Reservoir** — stores oil, lets air escape and heat dissipate.
- **Pump** — gear, vane or piston type; it creates *flow*, the load creates *pressure*.
- **Relief valve** — the safety fuse: dumps oil to tank if pressure exceeds the setting.
- **Valves** direct and meter the oil; **actuators** convert it into force and motion.
- Unlike pneumatics, oil is **returned** to the tank — it is a closed circuit.

Hydraulic Cylinder: Force and Speed

Key formulas

$$F = p A, \quad A = \frac{\pi D^2}{4}$$

$$v = \frac{Q}{A} \quad (Q = \text{flow rate})$$

$$P_{\text{hyd}} = p Q \quad (\text{power})$$

Force depends on **pressure**; speed depends on **flow**.

Example: bore $D = 100 \text{ mm}$, $p = 100 \text{ bar}$,
 $Q = 20 \text{ L/min}$.

$$A = \frac{\pi(0.1)^2}{4} = 7.854 \times 10^{-3} \text{ m}^2$$

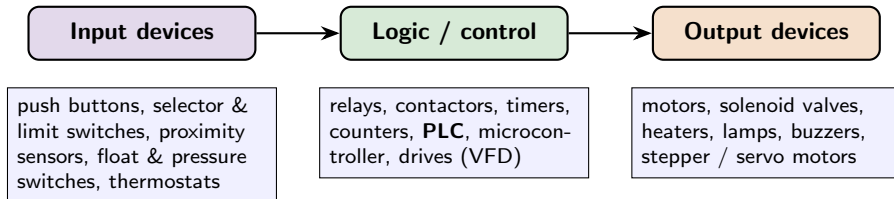
$$F = (100 \times 10^5)(7.854 \times 10^{-3}) \approx \boxed{78.5 \text{ kN}}$$

$$Q = \frac{20 \times 10^{-3}}{60} = 3.33 \times 10^{-4} \text{ m}^3/\text{s}$$

$$v = \frac{3.33 \times 10^{-4}}{7.854 \times 10^{-3}} \approx \boxed{42 \text{ mm/s}}$$

$$P = (10^7)(3.33 \times 10^{-4}) \approx 3.3 \text{ kW}$$

Elements of an Electrical Control System



Relay / contactor

An electrically operated switch: a small coil current (24 V) closes contacts that carry a large load current. A **contactor** is a heavy-duty relay for motors.

NO and NC contacts

NO (normally open) closes when operated; **NC** (normally closed) opens when operated. Stop buttons are always **NC** (fail-safe).

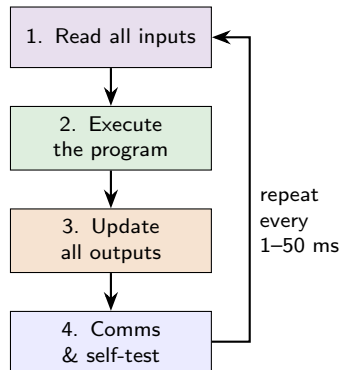
The Programmable Logic Controller (PLC)

PLC

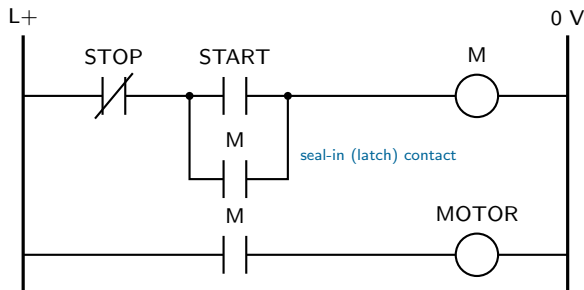
A rugged industrial computer that reads **inputs**, runs a user **program** and sets **outputs**. It replaced large panels of hard-wired relays.

- Programmed in IEC 61131-3 languages: **Ladder** (LD), Function Block (FBD), Structured Text (ST, looks like C/Pascal), ...
- Changing the logic = editing software, not rewiring.
- Built-in timers, counters, PID blocks, communication.

The PLC scan cycle



Ladder Logic: Motor Start/Stop (Seal-in) Circuit

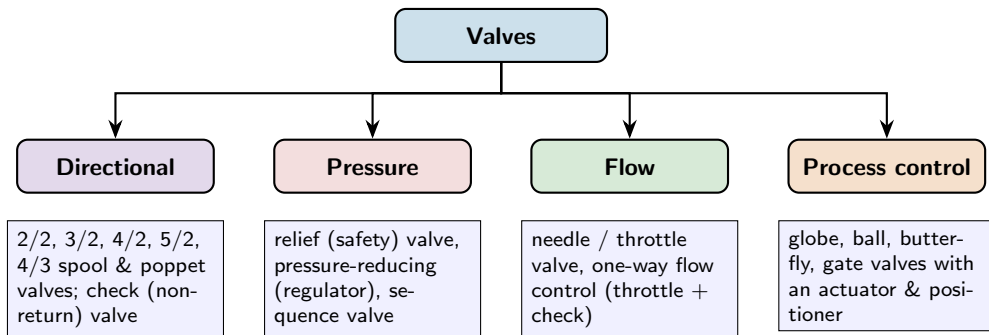


- Press **START** → coil **M** energises.
- Contact **M** in parallel keeps the coil ON after **START** is released — the circuit *remembers*.
- Press **STOP** (NC) → rung breaks, **M** drops out.
- Power failure also drops **M** — motor won't restart by itself (safety).

Same logic as one C statement:

```
M = (START || M) && !STOP;
```

Classification of Valves

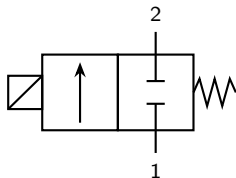


- **Directional** valves decide *where* fluid goes (ON/OFF — logic control).
- **Pressure** and **flow** valves decide *how hard* and *how fast*.
- **Process control** valves throttle continuously — they are the final control element of a PID loop.

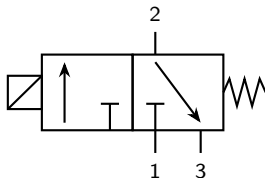
Directional Control Valves: Ports / Positions

Naming rule

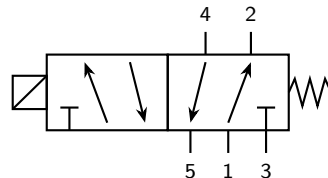
A valve is named “**ports / positions**”: a **3/2 valve** has 3 ports and 2 switching positions. Port numbers (ISO 5599): **1** = supply, **2, 4** = outputs, **3, 5** = exhausts.



2/2 NC



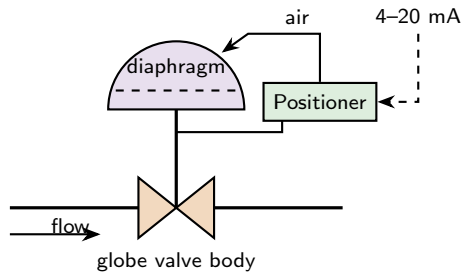
3/2 NC



5/2 single solenoid

- **2/2** — simple ON/OFF tap. **3/2** — drives a *single-acting* cylinder. **5/2** — drives a *double-acting* cylinder.
- Actuation: manual (button, lever), mechanical (roller), **solenoid**, pilot air; return usually by **spring**.

Process Control Valves



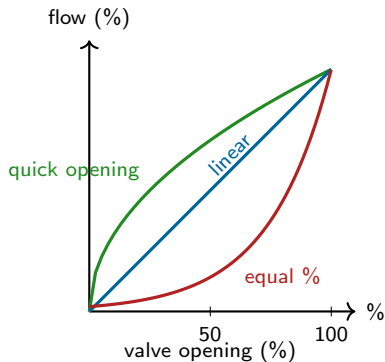
- **Body:** globe (fine throttling), ball, butterfly (large pipes), gate (isolation only).
- **Actuator:** diaphragm (air), piston, or electric motor moves the stem.
- **Positioner:** converts the 4–20 mA controller signal into accurate stem position.

Fail-safe action

Air-to-open (fail-closed) — e.g. fuel valve closes if air is lost.

Air-to-close (fail-open) — e.g. cooling-water valve opens if air is lost.

Control Valve Characteristics and Sizing



- **Linear:** $\text{flow} \propto \text{opening}$ — level loops.
- **Equal percentage:** each step adds the same % of current flow — most common for pressure / flow loops.
- **Quick opening:** nearly full flow at small opening — ON/OFF service.

Sizing: flow coefficient C_v

$$Q = C_v \sqrt{\frac{\Delta p}{SG}} \quad (\text{US gpm, psi})$$

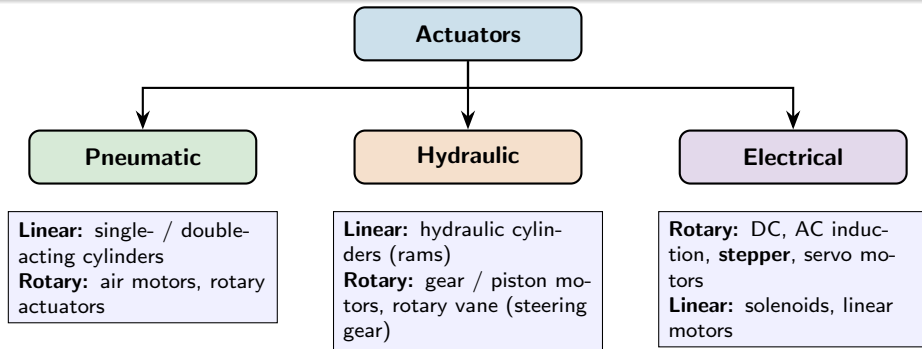
$C_v = 10$, $\Delta p = 16$ psi, water ($SG = 1$):

$$Q = 10\sqrt{16} = \boxed{40 \text{ gpm}}$$

Classification of Actuators

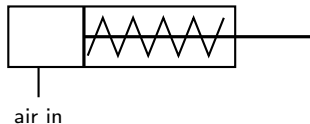
Actuator

A device that converts an energy source (air, oil, electricity) into **mechanical motion** — the “muscles” of an automated system.

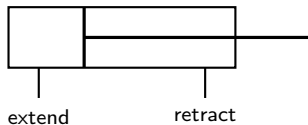


Linear Actuators: Cylinders

Single-acting (spring return)



Double-acting



Example: pneumatic cylinder, bore $D = 50$ mm, rod $d = 20$ mm, $p = 6$ bar (0.6 N/mm^2).

$$A_{\text{cap}} = \frac{\pi}{4}(50)^2 = 1963 \text{ mm}^2$$

$$F_{\text{ext}} = 0.6 \times 1963 \approx \boxed{1178 \text{ N}}$$

$$A_{\text{ann}} = \frac{\pi}{4}(50^2 - 20^2) = 1649 \text{ mm}^2$$

$$F_{\text{ret}} = 0.6 \times 1649 \approx \boxed{990 \text{ N}}$$

Retracting force is **smaller** because the rod takes up part of the piston area. (Real force $\approx 10\%$ less due to friction.)

Electrical Actuators at a Glance

Actuator	Principle	Control	Typical use
Solenoid	coil pulls an iron plunger	ON/OFF	valve pilots, door locks
DC motor	current in a magnetic field	voltage / PWM	fans, small drives, toys
AC induction motor	rotating magnetic field	VFD (frequency)	pumps, compressors, winches
Stepper motor	moves in fixed angular steps	pulses (open loop)	3D printers, valves, CNC
Servo motor	motor + encoder feedback	closed loop (PID)	robots, CNC axes

Key idea

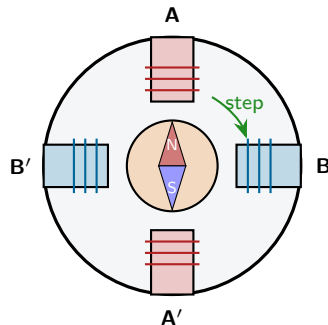
When **position** must be controlled precisely, we use either a **stepper** (count the pulses, no feedback needed) or a **servo** (measure the position and correct it with PID).

What is a Stepper Motor?

Stepper motor

A brushless motor that rotates in **discrete, equal angular steps**. Each input **pulse** moves the shaft by one **step angle**.

- Position = number of pulses $\times$ step angle.
- Speed $\propto$ pulse frequency.
- Direction set by the *order* in which coils are energised.
- Runs **open loop** — no position sensor needed (if not overloaded).
- Holds position firmly when stopped (**holding torque**).



Coil A energised $\rightarrow$ rotor aligns with A.
Next pulse energises B $\rightarrow$ rotor turns 90° .

Types of Stepper Motors

Permanent Magnet (PM)

- Rotor is a permanent magnet.
- Step angle 7.5° – 15° .
- Low cost, low speed; has detent torque.
- Printers, small valves.

Variable Reluctance (VR)

- Toothed soft-iron rotor, no magnet.
- Rotor moves to the position of **least reluctance**.
- No detent torque.
- $\theta = \frac{N_s - N_r}{N_s N_r} \times 360^{\circ}$

Hybrid

- Magnet + toothed rotor.
- Fine steps: 1.8° (200 steps/rev) or 0.9° .
- High torque & accuracy.
- CNC, 3D printers, robots.

Example (VR motor)

Stator teeth $N_s = 8$, rotor teeth $N_r = 6$: $\theta = \frac{8 - 6}{8 \times 6} \times 360^{\circ} = \boxed{15^{\circ}} \Rightarrow 360/15 = 24$ steps per revolution.

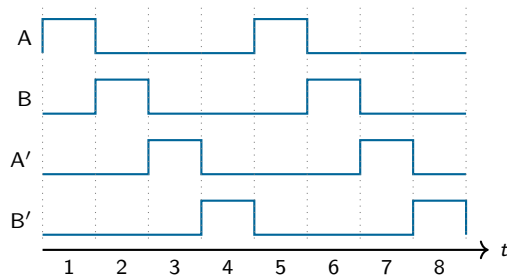
Drive Modes and Switching Sequence

Mode	Coils ON	Step
Wave drive	one at a time	full, low torque
Full step	two at a time	full, max torque
Half step	one & two alternately	half (0.9°)
Micro-step	sine/cosine currents	1/16, 1/256 ...

Full-step (two-phase ON) sequence

Step	A	B	A'	B'	Rotor
1	1	1	0	0	45°
2	0	1	1	0	135°
3	0	0	1	1	225°
4	1	0	0	1	315°

Wave-drive pulse timing



Reversing the order (B', A', B, A) reverses the direction.

Stepper Motor Calculations

Useful relations

$$\text{Steps/rev} = \frac{360^\circ}{\theta} \quad \text{Speed (rpm)} = \frac{f \theta}{6} \quad \text{Linear step} = \frac{\text{lead-screw pitch}}{\text{steps/rev}}$$

θ = step angle (degrees), f = pulse rate (pulses/s).

Example: hybrid motor, $\theta = 1.8^\circ$, driven at $f = 1000$ pulses/s.

$$\text{Steps/rev} = 360/1.8 = 200$$

$$\text{rev/s} = 1000/200 = 5$$

$$\text{Speed} = \frac{1000 \times 1.8}{6} = \boxed{300 \text{ rpm}}$$

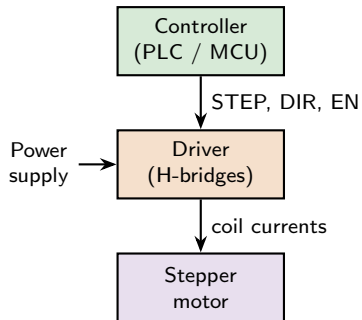
Example: same motor drives a lead screw of pitch 5 mm.

$$\text{Resolution} = 5/200 = \boxed{0.025 \text{ mm/step}}$$

$$\text{Pulses for 30 mm} = 30/0.025 = 1200$$

With 1/16 micro-stepping the resolution becomes 0.0016 mm.

Driving a Stepper Motor



The controller only sends **STEP** pulses and a **DIR** level; the driver switches the coil currents.

Advantages

Precise open-loop positioning, excellent holding torque, simple digital control, long life (no brushes).

Limitations

Torque falls at high speed; **missed steps** if overloaded (controller does not know!); resonance & vibration; draws current even at standstill.

Critical positioning (e.g. a ship's fuel rack) adds an encoder to detect lost steps.

Pseudocode

- 1 START
- 2 Set step angle = 1.8
- 3 READ target angle
- 4 $n = \text{angle} / 1.8$
(rounded)
- 5 PRINT n
- 6 FOR $i = 1$ to n
 - Output pattern
seq[i mod 4] to
coils A, B, A', B'
- 7 STOP

C Program (wave-drive sequence)

```
#include <stdio.h>

int main(void)
{
    float step = 1.8, angle;
    int    n, i;
    /* coil pattern: A B A' B' */
    const char *seq[4] =
        {"1000", "0100", "0010", "0001"};

    printf("Enter angle (deg): ");
    scanf("%f", &angle);

    n = (int)(angle / step + 0.5);
    printf("Steps needed = %d\n", n);

    for (i = 0; i < 8 && i < n; i++)
        printf("Step %d : %s\n", i+1, seq[i%4]);
    return 0;
}
```

Program Output

```
Enter angle (deg): 90
Steps needed = 50
Step 1 : 1000
Step 2 : 0100
Step 3 : 0010
Step 4 : 0001
Step 5 : 1000
Step 6 : 0100
Step 7 : 0010
Step 8 : 0001
```

Note: only the first 8 of the 50 patterns are printed. On a real controller each printf is replaced by writing the 4 bits to an output port, followed by a short delay that sets the speed.

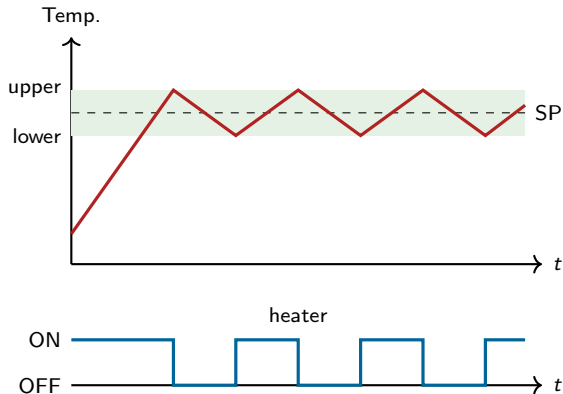
The Simplest Controller: ON-OFF

ON-OFF (two-position) control

Output is either **100 %** or **0 %**. A **dead band** (hysteresis) prevents rapid chattering.

- Example: fridge, water heater, bilge pump with float switch.
- Cheap and simple.
- But PV **always oscillates** around the setpoint, and the actuator wears out.

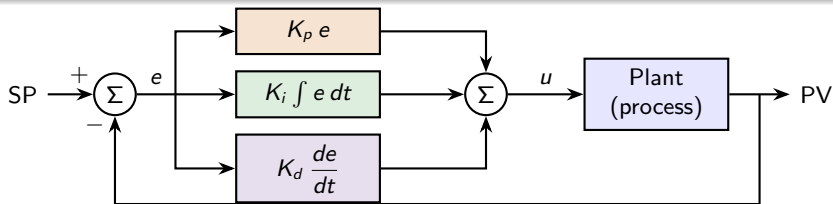
For smooth, accurate control we need an output that varies *continuously* with the error → **PID**.



The PID Controller

PID control law

$$u(t) = \underbrace{K_p e(t)}_{\text{Proportional}} + \underbrace{K_i \int_0^t e(\tau) d\tau}_{\text{Integral}} + \underbrace{K_d \frac{de(t)}{dt}}_{\text{Derivative}}, \quad e(t) = \text{SP} - \text{PV}$$



Industrial (standard) form: $u = K_p \left(e + \frac{1}{T_i} \int e dt + T_d \frac{de}{dt} \right), \quad K_i = \frac{K_p}{T_i}, \quad K_d = K_p T_d$

P, I and D Actions: What Each Term Does

P — Present

$u \propto$ **current error**

- Big error $\rightarrow$ big correction.
- Fast response.
- Leaves a steady-state **offset** for most loads.
- Proportional band: $PB = 100/K_p \%$.

I — Past

$u \propto$ **accumulated error**

- Keeps pushing while *any* error remains.
- **Eliminates offset.**
- Too much $\rightarrow$ overshoot, oscillation, *windup*.

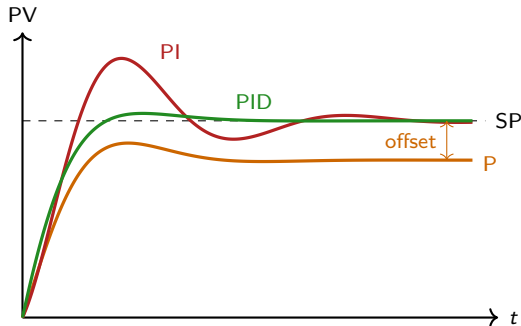
D — Future

$u \propto$ **rate of change**

- Anticipates where the error is heading.
- Adds **damping**, reduces overshoot.
- Amplifies sensor **noise** — often left out (PI).

Analogy — driving a car to a stop line: **P** brakes by distance left, **I** corrects if you keep stopping short, **D** eases off as you approach fast.

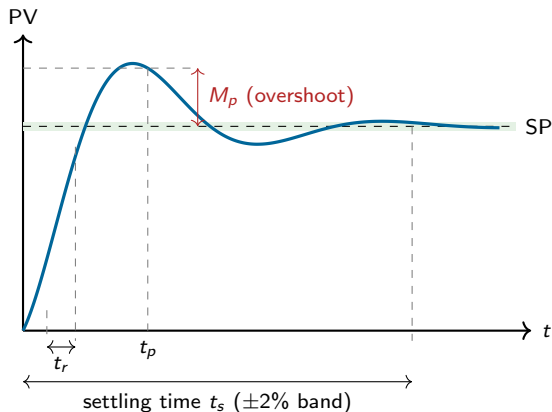
Response with P, PI and PID Control



Increase	Rise time	Over-shoot	Settling time	Steady error
K_p	↓	↑	small	↓
K_i	↓	↑	↑	removed
K_d	small	↓	↓	none

- **P**: fast, but settles *below* SP.
- **PI**: reaches SP, with overshoot.
- **PID**: reaches SP quickly and smoothly.

Measuring Performance: The Step Response



- **Rise time** t_r — 10 % to 90 % of final value.
- **Peak time** t_p — time to the first peak.
- **Overshoot** M_p —
$$\frac{\text{peak} - \text{final}}{\text{final}} \times 100 \%$$
- **Settling time** t_s — stays within $\pm 2\%$ (or 5 %).
- **Steady-state error** — SP – final PV.

Tuning = choosing K_p, K_i, K_d to meet these specifications.

Digital (Discrete-Time) PID

Computers sample every T seconds

Replace the integral by a **sum** and the derivative by a **difference**:

$$u[k] = K_p e[k] + K_i T \sum_{j=0}^k e[j] + K_d \frac{e[k] - e[k-1]}{T}$$

Example: $K_p = 2$, $K_i = 0.5$, $K_d = 1$, $T = 1$ s.

Errors so far: $e[0] = 4$, $e[1] = 3$, $e[2] = 1$.

$$P = 2 \times 1 = 2$$

$$I = 0.5 \times 1 \times (4 + 3 + 1) = 4$$

$$D = 1 \times \frac{1-3}{1} = -2$$

$$u[2] = 2 + 4 - 2 = \boxed{4}$$

The D term is *negative*: the error is falling fast, so the controller eases off to avoid overshoot.

Pseudocode

- 1 START
- 2 Set K_p, K_i, K_d, T, SP ; $PV = \text{sum} = e_{\text{prev}} = 0$
- 3 REPEAT every T
 - $e = SP - PV$
 - $\text{sum} += e * T$
 - $d = (e - e_{\text{prev}}) / T$
 - $u = K_p * e + K_i * \text{sum} + K_d * d$
 - apply u ; read PV ; $e_{\text{prev}} = e$
- 4 STOP after 5 s

Program Output

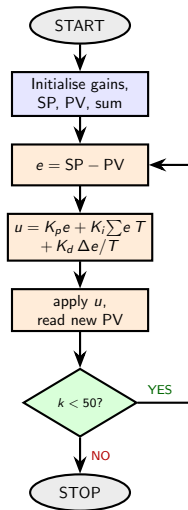
t=1.0 s PV=0.772
t=2.0 s PV=0.868
t=3.0 s PV=0.911
t=4.0 s PV=0.939
t=5.0 s PV=0.959

C Program (simulated loop)

```
#include <stdio.h>

int main(void)
{
    float Kp = 2, Ki = 1, Kd = 0.1;
    float T = 0.1;           /* sample time */
    float sp = 1, pv = 0;    /* SP and PV */
    float e, e_prev = 0, integ = 0, deriv, u;
    int k;

    for (k = 1; k <= 50; k++) {
        e = sp - pv;
        integ += e * T;
        deriv = (e - e_prev) / T;
        u = Kp*e + Ki*integ + Kd*deriv;
        pv += T * (u - pv); /* plant */
        e_prev = e;
        if (k % 10 == 0)
            printf("t=%.1f s PV=%.3f\n", k*T, pv);
    }
    return 0;
}
```



What is Controller Tuning?

Tuning

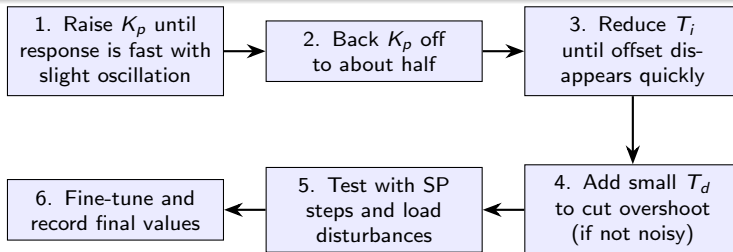
Choosing the controller parameters (K_p , T_i , T_d) so that the loop is **stable**, **fast** and has **acceptable overshoot** for *this particular* process.

Method	Idea	Needs
Manual (trial & error)	adjust P, then I, then D while watching the trend	experience, time
Ziegler–Nichols <i>closed loop</i>	push loop to sustained oscillation, use formulas	ultimate gain K_u , period P_u
Ziegler–Nichols <i>open loop</i>	step test of the process (reaction curve)	delay L , time constant T
Cohen–Coon, IMC, Lambda	model-based refinements	process model
Auto-tuning	controller runs its own test (relay method)	modern PLC / controller

Manual (Trial-and-Error) Tuning

Before you start

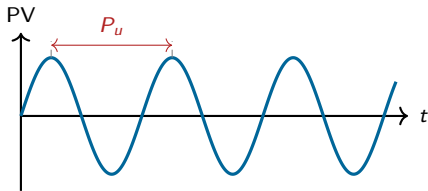
Put the controller in **AUTO** with $T_i = \infty$ (I off) and $T_d = 0$ (D off). Make small setpoint steps and watch the PV trend.



- Change **one** parameter at a time, by a factor of about 2.
- Many real loops (flow, level, pressure) work well as **PI** only.

Ziegler–Nichols Closed-Loop (Ultimate Gain) Method

- 1 Use **P-only** control ($T_i = \infty$, $T_d = 0$).
- 2 Increase K_p slowly until the PV oscillates with **constant amplitude**.
- 3 Record that gain K_u (*ultimate gain*) and the oscillation period P_u .
- 4 Read the settings from the table.



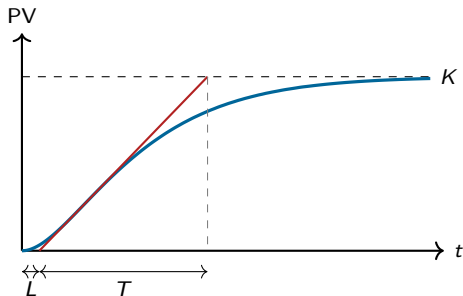
Type	K_p	T_i	T_d
P	$0.5 K_u$	∞	0
PI	$0.45 K_u$	$P_u/1.2$	0
PID	$0.6 K_u$	$P_u/2$	$P_u/8$

Caution

Driving a real plant to the edge of instability can be unsafe — the *relay auto-tune* method finds K_u , P_u with a small, controlled oscillation.

Ziegler–Nichols Open-Loop (Process Reaction Curve) Method

With the controller in **MANUAL**, make a step change in the output and record the PV. Draw the tangent at the steepest point:



L = dead time, T = time constant, K = process gain ($\Delta PV / \Delta CO$).

Type	K_p	T_i	T_d
P	$T / (KL)$	∞	0
PI	$0.9 T / (KL)$	$3.33 L$	0
PID	$1.2 T / (KL)$	$2 L$	$0.5 L$

Works for self-regulating processes that look like “first order + dead time” (temperature, pressure).

Tuning: Worked Examples

Closed-loop method

A P-only test gives sustained oscillation at $K_u = 10$ with period $P_u = 2$ s. Find PID settings.

$$K_p = 0.6 \times 10 = \boxed{6}$$

$$T_i = 2/2 = 1 \text{ s} \Rightarrow K_i = K_p/T_i = 6 \text{ s}^{-1}$$

$$T_d = 2/8 = 0.25 \text{ s} \Rightarrow K_d = K_p T_d = 1.5 \text{ s}$$

Open-loop method

A step test on a heater gives $K = 1$, dead time $L = 2$ s, time constant $T = 10$ s. Find PI and PID settings.

$$\text{PI: } K_p = 0.9 \times \frac{10}{1 \times 2} = \boxed{4.5}, \quad T_i = 6.67 \text{ s}$$

$$\text{PID: } K_p = 1.2 \times \frac{10}{2} = \boxed{6}$$

$$T_i = 2 \times 2 = 4 \text{ s}, \quad T_d = 0.5 \times 2 = 1 \text{ s}$$

Ziegler–Nichols values give a quick, “aggressive” response ($\approx 25\%$ overshoot) — treat them as a **starting point** and fine-tune.

Practical Issues in Real PID Loops

Integral windup

When the valve is already 100 % open, the integral keeps growing; when the error reverses, it takes a long time to “unwind” → big overshoot.

Fix: clamp the integral (anti-windup).

Derivative kick

A sudden SP step makes de/dt huge → output spike.

Fix: take the derivative of the *PV*, not the error.

Measurement noise

D action amplifies noise.

Fix: filter the *PV*, or use PI only.

Bumpless transfer

Switching MANUAL ↔ AUTO must not make the output jump.

Fix: initialise the integral to the current output.

Controller action

Reverse (heating: $PV \uparrow \Rightarrow \text{output} \downarrow$) or **direct** (cooling). Wrong choice = runaway loop!

Summary & What Comes Next

Concept

- Industrial processes: continuous, batch, discrete; automation levels.
- Control loop elements: sensor, controller, final control element.
- Pneumatic, hydraulic & electrical systems; PLC and ladder logic.
- Valves (directional, pressure, flow, control) and actuators.
- Stepper motors: types, step angle, drive sequences.
- PID control and its tuning (manual, Ziegler–Nichols).

Practice makes perfect

- Re-work every numerical example yourself.
- Draw the valve symbols from memory.
- Run the PID C program; change K_p , K_i , K_d and watch the PV.

Coming up

PLC programming, sensors & transducers, and marine automation systems.

Thank You

Questions?